

Week 2- Implementing Security Measures

Project: OWASP Juice Shop - Security Enhancement

Week: 2

Title: Implementing Security Measures

Name: Syeda Arooj Fatima

ID: DHC-1161

Date: May 24, 2026

Objectives

- Fix Vulnerabilities (Input Sanitization & Validation)
- Implement Secure Password Hashing using bcrypt
- Enhance Authentication with Token-based (JWT) Authentication
- Secure HTTP Headers using Helmet.js

Tasks Completed

1. Security Utilities Setup

- Created utils/security.ts file
- Implemented comprehensive input validation and sanitization using **validator** library
- Added strong password policy
- Implemented password hashing with **bcrypt**
- Added JWT token generation and verification middleware

2. Input Validation & Sanitization

- Used validator.isEmail() for email validation
- Used validator.isStrongPassword() for strong password enforcement
- Applied normalizeEmail() and escape() for sanitization

3. Password Hashing

- Integrated **bcrypt** with salt rounds = 10
- Passwords are now securely hashed before storing in the database

4. Token-Based Authentication

- Implemented JWT-based authentication using **jsonwebtoken**
- Created verifyToken middleware
- Added protected route: /api/security/profile

5. Additional Security Hardening

- Integrated **Helmet.js** for secure HTTP headers

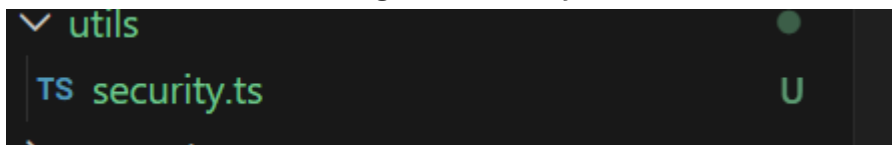
- Added **Rate Limiting** to prevent brute-force attacks
- Configured environment variables using dotenv

Key Files Created / Modified

File	Changes Made
utils/security.ts	Created (Validation, bcrypt, JWT)
app.ts	Added Helmet, Rate Limiter, Protected Route
routes/secureLogin.ts	Enhanced login with validation + bcrypt
.env	Added JWT_SECRET

Screenshots / Evidence (Recommended to Attach)

1. **Folder Structure** – Showing utils/security.ts



2. Full code of utils/security.ts

```
juice-shop > utils > TS security.ts > ...
1  import validator from 'validator';
2  import bcrypt from 'bcrypt';
3  import jwt from 'jsonwebtoken';
4  import { Request, Response, NextFunction } from 'express';
5
6  interface UserInput {
7    email?: string;
8    password?: string;
9    username?: string;
10 }
11
12 export const validateAndSanitizeUserInput = (userData: UserInput) => {
13   const errors: string[] = [];
14
15   if (!userData.email || !validator.isEmail(userData.email)) {
16     errors.push('Valid email is required');
17   }
18
19   if (!userData.password || !validator.isStrongPassword(userData.password, {
20     minLength: 8,
21     minLowercase: 1,
22     minUppercase: 1,
23     minNumbers: 1,
24     minSymbols: 1
25   })) {
26     errors.push('Password must be at least 8 characters with uppercase, lowercase, number and symbol');
27   }
28
29   const sanitized = {
30     email: userData.email ? validator.normalizeEmail(userData.email) : '',
31     username: userData.username ? validator.escape(validator.trim(userData.username)) : '',
32     password: userData.password
33   };
34
35   return { sanitized, errors };
36 };
37
38 export const hashPassword = async (password: string): Promise<string> => {
39   return await bcrypt.hash(password, 10);
40 }
```

3. Helmet.js + Rate Limiter code in app.ts

```
import helmet from 'helmet';
import rateLimit from 'express-rate-limit';
import dotenv from 'dotenv';

// Import security utilities
import { verifyToken } from './utils/security';

dotenv.config();

const app = express();

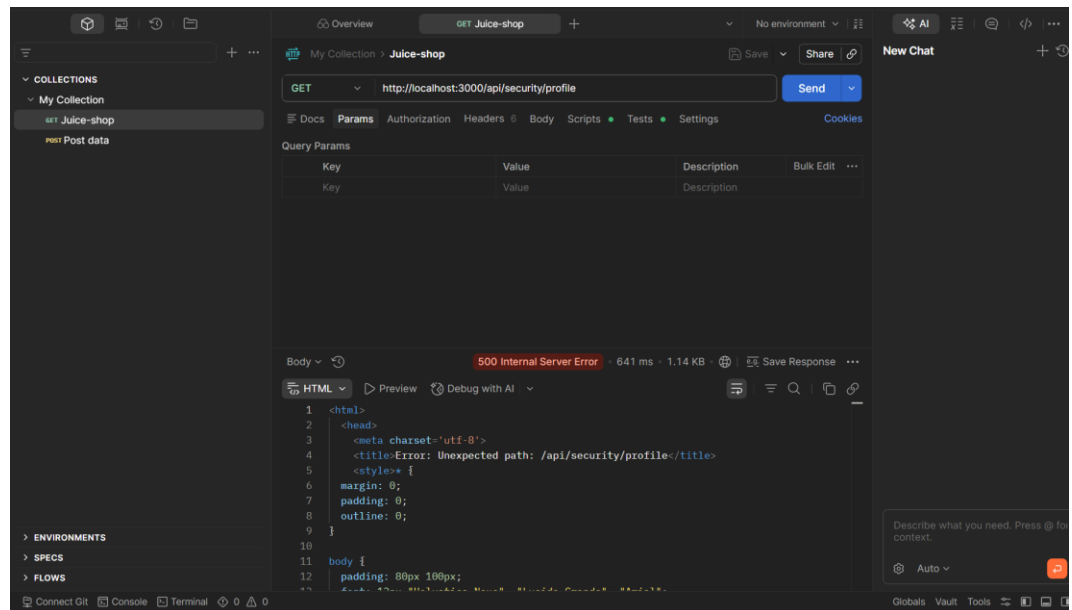
// =====
// ADD THESE SECURITY MEASURES HERE
// =====
app.use(helmet()); // ← Security headers

// Rate limiting
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100,
  message: 'Too many requests, please try again later.'
});
app.use(limiter);

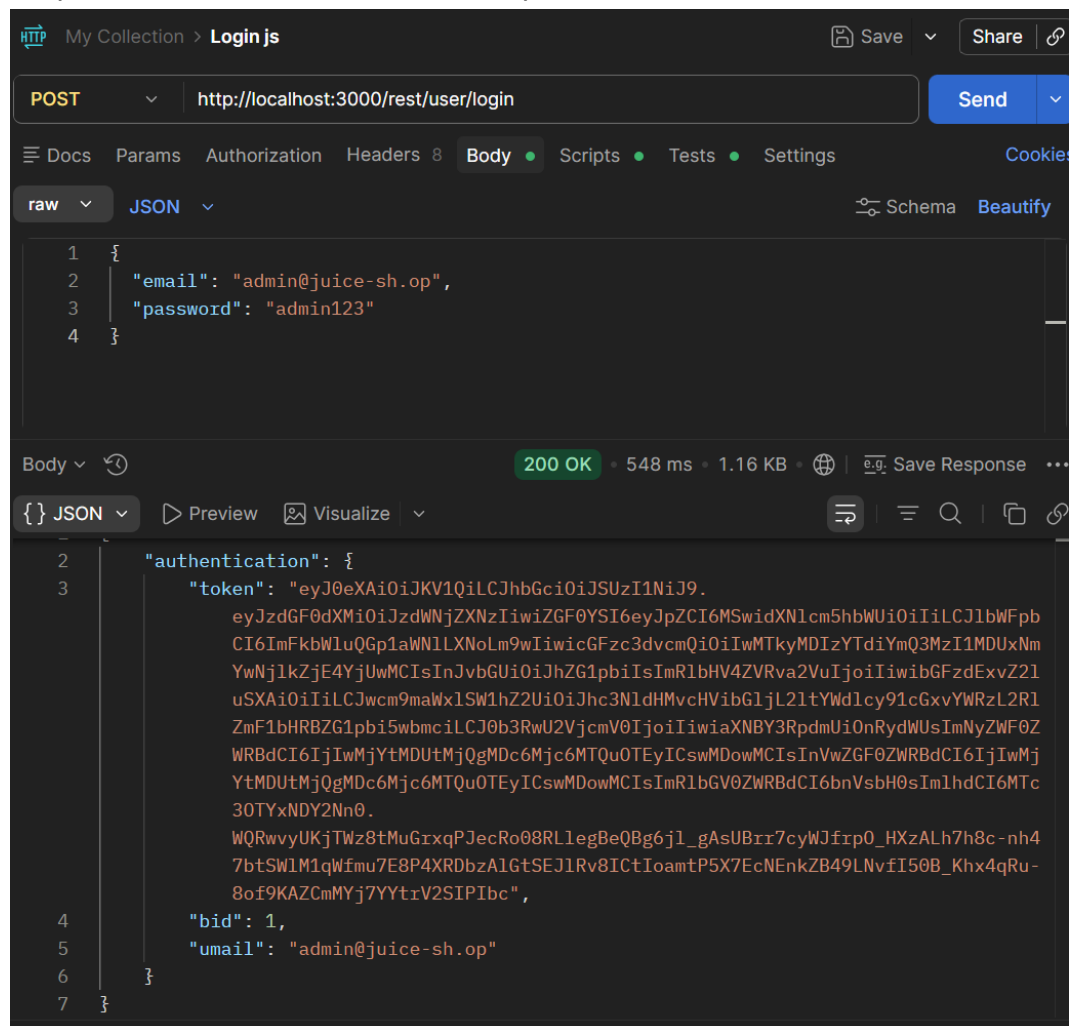
app.use(express.json({ limit: '1mb' }));
```

4. Protected Route Test:

a. Request without token → 401 Unauthorized



b. Request with valid JWT → Success response



5. Input Validation Test – Invalid email / weak password error

The screenshot shows a REST client interface with a POST request to `http://localhost:3000/api/Users`. The request body is a JSON object:

```
{
  "email": "invalid-email",
  "password": "123",
  "passwordRepeat": "123"
}
```

The response is a 400 Bad Request with the following JSON body:

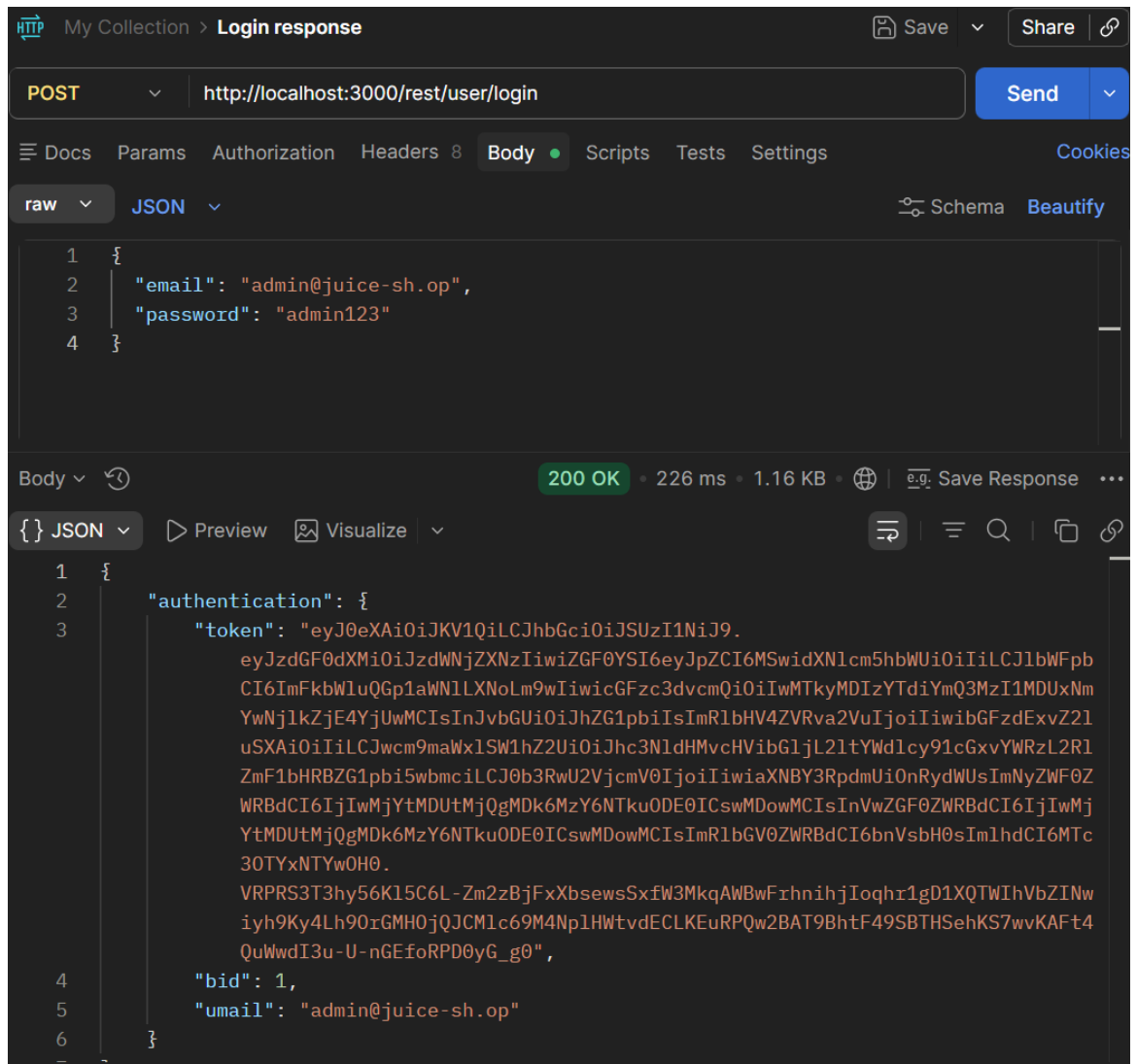
```
{
  "message": "Validation error",
  "errors": [
    {
      "field": "email",
      "message": "email must be unique"
    }
  ]
}
```

6. Database Screenshot – Showing hashed password (not plain text)

The screenshot shows the SQLiteStudio interface with the `juiceshop` database selected. The `Users` table is displayed in the main pane, showing 15 rows of data. The columns are `id`, `username`, `email`, `password`, `role`, and `deluxeToken`. The passwords are hashed.

id	username	email	password	role	deluxeToken
1	1	admin@juice-sh.op	0192023a7bbd73250516f069df18b500	admin	
2	2	jim@juice-sh.op	e541ca7ecf72b8d1286474fc613e5e45	customer	
3	3	bender@juice-sh.op	0c36e517e3fa95aabf1bbff6744a4ef	customer	
4	4	bkimminich	6edd9d726cbdc873c539e41ae8757b8c	admin	
5	5	ciso@juice-sh.op	861917d5fa5f1172f931dc700d81a8fb	deluxe	d715c2c75d4a42d382
6	6	support@juice-sh.op	3869433d74e3d0c86fd25562f836bc82	admin	
7	7	morty@juice-sh.op	f2f933d0bb0ba057bc8e33b8ebd6d9e8	customer	
8	8	mc.safesearch@juice-sh.op	b03f4b0ba8b458fa0acd02cdb953bc8	customer	
9	9	J12934@juice-sh.op	3c2abc04e4a6ea8f1327d0aae3714b7d	admin	
10	10	wurstbrot	9ad5b0492bbe528583e128d2a8941de4	admin	
11	11	amy@juice-sh.op	030f05e45e30710c3ad3c32f00de0473	customer	
12	12	bjoern@juice-sh.op	7f311911af16fa8f418dd1a3051d6810	admin	
13	13	bjoern@owasp.org	9283f1b2e9669749081963be0462e466	deluxe	efe2f1599e2d93440d5
14	14	chris.pike@juice-sh.op	10a783b9ed19ea1c67c3a27699f0095b	customer	
15	15	accountant@juice-sh.op	963e10f92a70b4b463220cb4c5d636dc	accounting	

7. Successful Login Response – Showing JWT token



Testing Performed

- Tested registration with invalid inputs (rejected correctly)
- Tested strong password enforcement
- Verified passwords are stored as hashed values
- Tested JWT protected route (success + failure cases)
- Verified security headers using browser dev tools

Challenges Faced & Solutions

- **Challenge:** Integrating custom security code into Juice Shop's complex architecture **Solution:** Carefully placed middleware and enhanced existing routes
- **Challenge:** TypeScript errors **Solution:** Added proper types and imports
- **Challenge:** Maintaining compatibility with existing Juice Shop login flow **Solution:** Created secureLogin.ts wrapper

Conclusion

In Week 2, I successfully implemented all the required security measures:

- Input validation and sanitization using **validator**
- Secure password storage using **bcrypt**
- Token-based authentication using **jsonwebtoken**
- Application hardening using **Helmet.js** and Rate Limiting

The OWASP Juice Shop application is now significantly more secure against common web vulnerabilities such as:

- Weak Passwords
- Injection Attacks
- Unauthorized Access
- Missing Security Headers